

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I R.Jaideep(192525441) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Jaideep

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS-Based Storage Solution for a Media Platform

Introduction

A media platform stores large amounts of videos, images, audio files, thumbnails, and metadata. Since the data volume continuously increases, HDFS is suitable because it provides distributed storage, fault tolerance, and scalable capacity.

Proposed Design

The HDFS cluster can be designed with:

- **Name Node:** Maintains the HDFS namespace and metadata.
- **Secondary Name Node/Checkpoint Node:** Performs checkpointing of filesystem metadata.
- **Data Nodes:** Store actual media blocks.
- **Replication Factor:** Set to 3, which is the maximum allowed.
- **Block Size:** Use a large block size such as **256 MB or 512 MB** for large media files to reduce metadata overhead.
- **Rack Awareness:** Place replicas across different racks to protect against rack-level failures.
- **High Availability:** Use an Active and Standby Name Node configuration where possible.

Data Availability

A replication factor of 3 means that every HDFS block has three copies. The replicas are distributed across different Data Nodes and racks. If one Data Node fails, another replica can immediately serve the data.

This provides high fault tolerance and helps maintain the required **more than 99.9% availability**.

Cost Efficiency

The replication factor cannot exceed 3, so unnecessary copies are avoided. Large block sizes reduce metadata overhead and improve sequential media processing. Older or less frequently accessed media can also be moved to lower-cost storage using an appropriate storage policy.

Justification

This design satisfies all constraints:

- **Availability:** Three replicas and rack awareness protect against node and rack failures.
- **Replication limit:** Replication factor is exactly 3.
- **Cost efficiency:** No replication beyond the required level and efficient block sizing reduce storage overhead.
- **Scalability:** New Data Nodes can be added as the media collection grows.

Therefore, HDFS with replication factor 3, rack awareness, and Name Node high availability is an appropriate solution for the media platform.

2. MapReduce Workflow for Processing Logs Within a 20-Minute Batch Window

Introduction

Log processing involves large volumes of records generated by servers, applications, and network devices. The MapReduce job must complete within a **20-minute batch window** while operating with a limited number of cluster nodes.

Proposed Job Design

The workflow can contain the following stages:

Log Files → Input Splitting → Mapper → Shuffle and Sort → Reducer → Output

Mapper

The Mapper reads each log record and extracts useful information such as:

- Timestamp
- IP address
- User ID
- HTTP status code
- Requested URL
- Error information

For example, the mapper can generate:

`<status_code, 1>`

for each log entry.

Shuffle and Sort

The MapReduce framework groups all records having the same key. For example, all records with HTTP status code 404 are grouped together.

Reducer

The Reducer aggregates the values:

`<404, [1,1,1,1,...]> → <404, total_count>`

This produces the total number of occurrences of each status code.

Task Distribution

Since the number of cluster nodes is limited, the input should be divided into an appropriate number of splits. Multiple mapper tasks can process different input splits in parallel.

For example, with four worker nodes:

- Node 1 → Mapper tasks for input splits 1 and 2
- Node 2 → Mapper tasks for input splits 3 and 4
- Node 3 → Mapper tasks for input splits 5 and 6
- Node 4 → Reducer and remaining mapper tasks

The actual task placement is decided by YARN based on available resources.

Techniques to Meet the 20-Minute Constraint

1. Use appropriate HDFS block sizes.
2. Run mapper tasks in parallel.
3. Minimize unnecessary data transferred during shuffle.
4. Use a **Combiner** to perform local aggregation before data is transferred to reducers.
5. Avoid unnecessary reducers.
6. Use data compression to reduce network traffic.
7. Prefer data-local execution so that mappers process data close to where it is stored.
8. Allocate sufficient memory and CPU to MapReduce containers.

Justification

Parallel mapper execution reduces processing time, while combiners reduce shuffle traffic. Data locality minimizes network overhead. A carefully selected number of reducers prevents reducer bottlenecks.

Thus, the proposed MapReduce workflow can efficiently process large log batches within the **20-minute processing window**, even with a limited number of cluster nodes.

3. YARN Scheduler Policy for ETL, Reporting, and Machine Learning Jobs

Introduction

YARN manages CPU, memory, and other cluster resources among different applications. In this case, the cluster must support ETL, reporting, and machine learning workloads while maintaining fairness and giving ETL jobs higher priority.

Proposed Scheduler

A **Capacity Scheduler** can be used with separate queues:

Queue	Priority	Purpose
ETL	High	Data ingestion and transformation
Reporting	Medium	Business reports and analytics
Machine Learning	Normal/Low	Model training and experimentation

Resource Allocation

A possible allocation is:

- **ETL:** 50% of cluster capacity
- **Reporting:** 25%
- **Machine Learning:** 25%

The exact values can be adjusted according to workload requirements.

ETL Priority

ETL jobs receive higher priority because downstream reporting and machine learning jobs depend on processed data. The ETL queue can therefore receive resources preferentially when demand is high.

However, resource limits should be maintained so that reporting and machine learning workloads are not completely starved.

Fairness

Fairness can be maintained by allowing unused resources from one queue to be temporarily used by other queues. When the original queue requires those resources, they can be reclaimed according to scheduler policies.

Within each queue, applications can receive fair resource allocation.

Node Failure Recovery

YARN ResourceManager monitors applications and cluster resources. If a NodeManager fails:

1. The failure is detected through heartbeats.
2. Containers running on the failed node are considered lost.

3. YARN reschedules the failed tasks on healthy nodes.
4. The application continues processing from available data and checkpoints.
5. Failed nodes can rejoin the cluster after recovery.

Justification

The proposed policy provides:

- **Fairness:** Resources are shared among different workloads.
- **ETL priority:** ETL receives higher scheduling priority.
- **Resource utilization:** Idle resources can be used by other queues.
- **Failure recovery:** Failed tasks are automatically rescheduled.

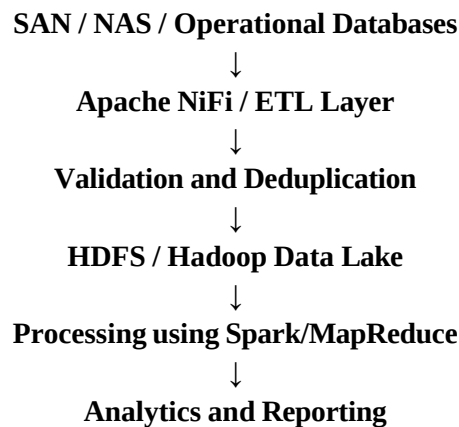
Therefore, a queue-based YARN scheduling policy provides a good balance between priority, fairness, and reliability.

4. Enterprise Data Ingestion from SAN, NAS, and Operational Databases

Introduction

An enterprise may have data distributed across SAN storage, NAS systems, and operational databases. The big data platform should ingest this data reliably while avoiding unnecessary duplication.

Proposed Architecture



Components

1. SAN

SAN contains high-performance enterprise storage. Data can be transferred through suitable file-system or storage interfaces into the ingestion layer.

2. NAS

NAS files can be collected using file-based ingestion mechanisms. Apache NiFi can monitor directories and transfer new or modified files.

3. Operational Databases

Database data can be ingested incrementally using database connectors. Instead of repeatedly copying the entire database, the system should use timestamps, transaction IDs, or change-data-capture mechanisms.

4. Apache NiFi / ETL Layer

NiFi acts as the central ingestion and flow-management layer. It provides:

- Data routing
- Data transformation
- Back-pressure
- Monitoring
- Error handling
- Flow control

5. Deduplication

Unique record IDs, file hashes, timestamps, or database transaction identifiers can be used to identify duplicate data. This prevents the same information from being stored multiple times.

6. HDFS

HDFS provides distributed storage and fault tolerance. Important data can use replication factor 3.

7. Backup

Backup can be implemented using HDFS snapshots, replication to another cluster, or a suitable enterprise backup system.

Justification

This architecture separates ingestion, processing, and storage. NiFi provides controlled data movement, while incremental database ingestion reduces unnecessary data transfer. Deduplication reduces storage consumption, and HDFS replication plus backup mechanisms improve reliability.

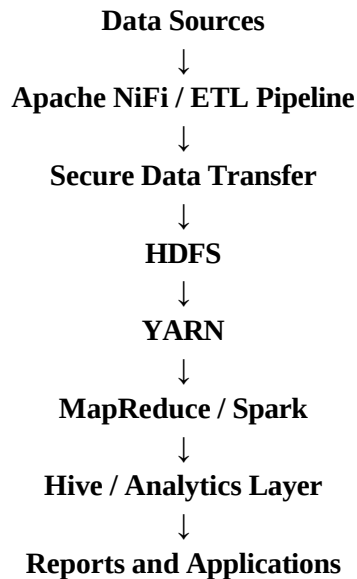
Therefore, the architecture provides reliable ingestion from heterogeneous enterprise sources while controlling duplication and maintaining data protection.

5. End-to-End Hadoop Ecosystem Solution with Secure Data Movement and High Availability

Introduction

A complete Hadoop ecosystem should provide secure data ingestion, reliable storage, processing, resource management, and integration with ETL tools such as Apache NiFi.

Proposed Architecture



1. Secure Data Movement

Apache NiFi can be used for controlled data ingestion from different sources. Security can be provided through:

- TLS/SSL encryption
- Authentication
- Authorization
- Secure credentials
- Access control policies

Data should be encrypted during transfer so that sensitive information cannot be intercepted.

2. High Availability

HDFS should use:

- Active and Standby NameNodes
- Multiple DataNodes
- Replication factor of 3
- Rack-aware replica placement

YARN can use ResourceManager high availability so that resource management continues even if the active ResourceManager fails.

3. Data Storage

HDFS stores large datasets in blocks distributed across multiple DataNodes. Replication provides fault tolerance. Important datasets can be protected using snapshots and backup mechanisms.

4. Processing

YARN manages cluster resources and schedules processing applications.

- **MapReduce:** Suitable for large batch processing.
- **Spark:** Suitable for faster iterative processing and machine learning.
- **Hive:** Provides SQL-based analysis of Hadoop data.

5. Apache NiFi / ETL Integration

NiFi receives data from databases, files, APIs, and other enterprise sources. It can then:

1. Receive the data.
2. Validate the data.
3. Transform the data.
4. Remove duplicates.
5. Encrypt or secure the transfer.
6. Store the data in HDFS.
7. Monitor the complete data flow.

6. Monitoring and Recovery

The Hadoop ecosystem should continuously monitor DataNodes, NodeManagers, NameNodes, and applications. When a node fails, replicated data remains available and failed processing tasks can be rescheduled on healthy nodes.

Constraint Satisfaction

Constraint	Design Solution
------------	-----------------

Secure data movement	NiFi + TLS/SSL + authentication
----------------------	---------------------------------

High availability	NameNode HA + ResourceManager HA + HDFS replication
-------------------	---

Reliable storage	HDFS with replication factor 3
------------------	--------------------------------

ETL integration	Apache NiFi and ETL pipelines
-----------------	-------------------------------

Large-scale processing	MapReduce/Spark
------------------------	-----------------

Constraint	Design Solution
Resource management	YARN
Data analysis	Hive/Spark
Failure recovery	Replication and task rescheduling

Conclusion

The proposed end-to-end Hadoop architecture integrates Apache NiFi with HDFS, YARN, MapReduce/Spark, and Hive. Secure communication protects data during movement, HDFS replication and Hadoop high-availability mechanisms protect against failures, and NiFi provides reliable ETL integration. Hence, the architecture satisfies the requirements of **secure data movement, high availability, scalable storage, and ETL integration**.